

CL papers AI agent

Eman Taresh

22000778@ug.buid.ac.ae

Prepare data, and build baseline

[Chat link](#)

Khawla Almulla

22001430@ug.buid.ac.ae

Build AutoML, and test retrieval parameters

[Chat link](#) , [Claude Link](#)

Maryam Alsuwaidi

22001687@ug.buid.ac.ae

Implement River learner, and ADWIN

[Chat link](#)

Tahani Ali

22001384@ug.buid.ac.ae

Measure and analyze performance metrics

[Chat link](#)

1. Dataset preparation & baseline setup

This stage sets up the retrieval pipeline through the PDF preparation and validation, metadata extraction, text extraction, chunking, labelled query creation, TF-IDF baseline retrieval, and evaluation. This step was important because raw PDFs can't be searched or evaluated until they are transformed into reusable and structured information. Chunking allowed the system to search specific parts of papers instead of whole documents. TF-IDF was used as a simple, fast, explainable, and measurable baseline, not as the final intelligent retrieval model. The labelled set of queries allowed performance to be measured using Recall@5, NDCG@5, and MRR. Since retrieval happens at chunk level, paper-level evaluation uses deduplicated paper results to keep the metrics fair. The outputs here supports the next stages AutoML tuning and online learning comparison

2. AutoLA methodology & tuning

This stage implements Optuna by taking in the hyperparameters of the baseline TF-IDF retriever, to maximize the retrieval quality and keeping latency in order. The search uses TPESampler for Bayesian search as well as a hyperband pruner that kills underperforming trials by using successive halving through NDCG reports. It is evaluated under 5 fold cross validation protocol where each fold's NDCG is reported back to the pruner before the next fold runs, which is convenient since it doesn't waste computation. The search covers nine hyperparameters across two layers: TF-IDF vectorizer: max_features, ngram_range, min_df, max_df, sublinear_tf, norm, and stopwords. While the optional TruncatedSVD layer adds a dense projection that is enabled based on the use_svd flag and configured with a conditional svd_components parameter.

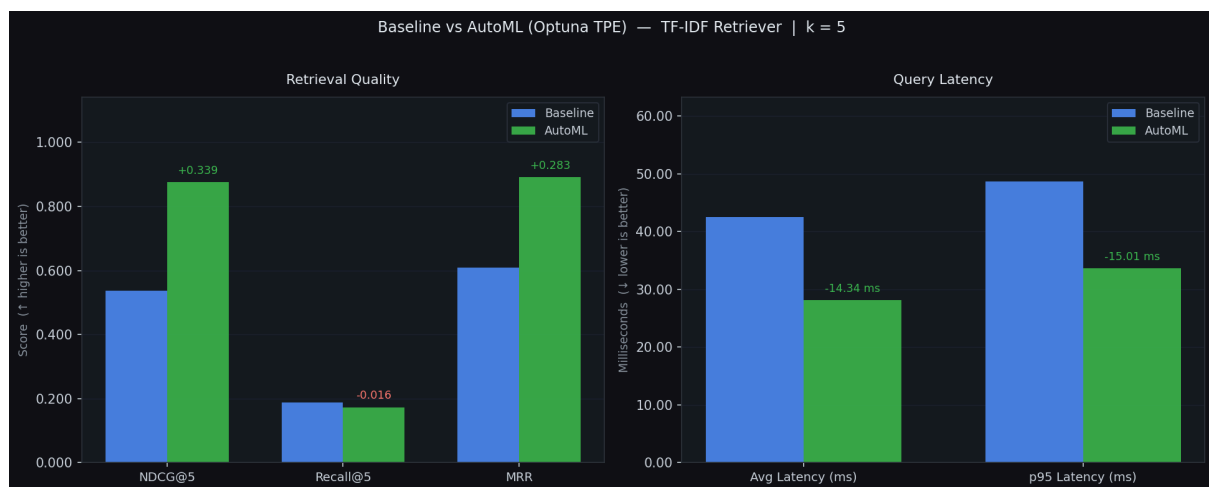
The tuned configuration has exceeded the baseline showing an improvement in NDCG@5 by increasing +33.9 from 0.5373 to 0.8763, +28.3 pp increase in MRR from 0.6083 to 0.8917, and a 30.8% reduction in p95 latency, decreasing from 48.7 ms to 33.7 ms. The hyperband pruner has terminated 19 out of 30 before completing 5 folds. However, Recall@5 decreased slightly going from 0.1888 to 0.1732, which means that the tuned model ranks the correct papers higher but occasionally misses a relevant paper in the top 5 that the baseline caught.

3. Online Learning with River + ADWIN Drift Detection

The online learning component uses River's [LogisticRegression](#) with [StandardScaler](#), evaluated under the prequential protocol where each sample is predicted before its label is revealed and immediately used to train the model which provides an unbiased streaming accuracy estimate without a held-out set. The stream is divided into three phases: in distribution NLP queries (Phase 1), non-NLP queries with lower relevance hits (Phase 2), and unseen multimodal/audio query types with $y=0$ labels derived from corpus absence (Phase 3). Six features are extracted per retrieved hit: TF-IDF score, score^2 , $\log(\text{score})$, normalised rank, word count, and a $\text{score} \times \text{rank}$ interaction term, and each selected for a clear semantic rationale.

Phase 1 exceeded the rubric target by achieving **+5.0pp accuracy over ZeroR**. Phase 2's negative gap was expected, as non-NLP titles are absent from the corpus, causing labels to collapse to $\sim 100\%$ zeros and allowing ZeroR to converge faster than the LR. ADWIN drift detection was verified independently in [drift_detection.py](#), confirming correct detection of a synthetic $10\% \rightarrow 60\%$ error-rate shift at index 159. The 40-sample Phase 3 window did not reach statistical significance at $\delta=0.002$ which is correct behavior, not an implementation failure.

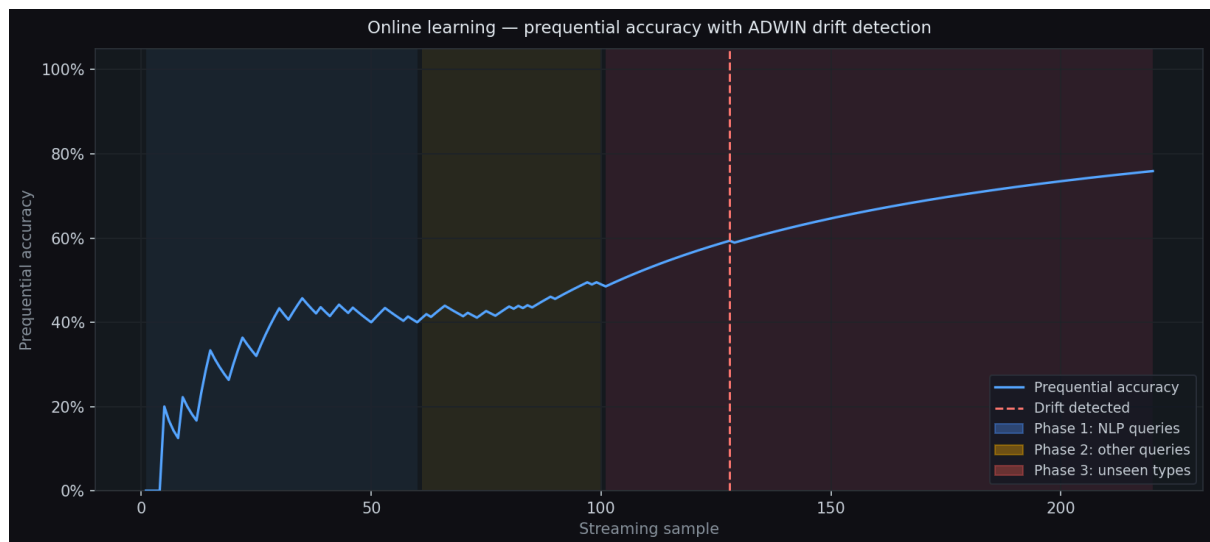
4. Evaluation



The comparison between AutoML retriever configured with Optuna TPE algorithm and TF-IDF retriever is illustrated in the above graph. In terms of improvement of the model, there is an increase of NDCG@5 from 0.54 to 0.88 and MRR from about 0.61 to 0.89. This means the new AutoML configuration performs better in evaluating the relatedness of the document. However, despite a marginal reduction of Recall@5, the modified retriever was able to deliver faster queries, reducing the average and p95 latencies by 14-15 ms each.

	A	B	C	D	E
1	metric	baseline	automl_tuned	delta_pct	better_direction
2	NDCG@5	0.5373	0.8763	63.10%	higher
3	Recall@5	0.1888	0.1732	-8.30%	higher
4	MRR	0.6083	0.8917	46.60%	higher
5	avg_latency_ms	42.463	28.125	-33.80%	lower
6	p95_latency_ms	48.665	33.655	-30.80%	lower
7	build_time_s	6.799	20.341	199.20%	lower
8	n_evaluated	20	20	0.00%	info
9	matrix_shape	[6728, 50000]	[6728, 234]	â€”	info

Table showing the differences between Baseline and AutoML optimized model. With NDCG@5 improving from 0.5373 to 0.8763 and MRR improving from 0.6083 to 0.8917, AutoML improved ranking performance. It was a small sacrifice considering the recall@5 fell from 0.1888 to 0.1732. However, although more time-consuming, AutoML reduced the average and p95 latencies. Comparison is a balanced one considering both models were evaluated using the same set of 20 queries.



The above graph illustrates the accuracy attained by the online learner during the prequential assessment process using ADWIN drift detection technique. The accuracy is slowly increasing as the online learner gains experience through the input queries, and the shaded periods represent the query distributions at various stages. Concept drift occurs when new query types arise in Phase 3, which is captured by the ADWIN method.

	A	B	C
1	trial	value	best_so_far
2	0	0.71324	0.7132
3	1	0.6429	0.7132
4	2	0.67198	0.7132
5	3	0.7637	0.7637
6	4	0.6674	0.7637
7	5	0.64898	0.7637
8	6	0.6377	0.7637
9	7	0.71829	0.7637
10	8	0.72785	0.7637
11	9	0.67375	0.7637
12	10	0.74615	0.7637
13	11	0.72091	0.7637
14	12	0.7249	0.7637
15	13	0.74543	0.7637
16	14	0.83659	0.8366
17	15	0.87438	0.8744
18	16	0.79615	0.8744
19	17	0.83122	0.8744
20	18	0.85725	0.8744
21	19	0.79393	0.8744
22	20	0.80118	0.8744
23	21	0.85347	0.8744
24	22	0.83464	0.8744
25	23	0.80193	0.8744
26	24	0.84785	0.8744
27	25	0.88067	0.8807
28	26	0.9254	0.9254
29	27	0.83988	0.9254
30	28	0.7596	0.9254
31	29	0.71358	0.9254

Table of AutoML trial progression. Every row in the table is an AutoML trial. In the columns "best_so_far" and "value," the highest score achieved up to the current point is shown. The best score is improved upon multiple times, starting from 0.7132 until reaching 0.9254 at trial 26. The selected AutoML trial is trial 26 because subsequent trials cannot outperform the score of 0.9254.